

Il Lato Oscuro della Variable Importance

Instabilità e Bias nella Selezione di Variabili in Random Forest

Statistical Learning – A.A. 2024/25

2026-05-20

Contents

1	Fondamenti teorici	1
1.1	Il contesto: perché questo problema è rilevante	1
1.2	Random Forest: una sintesi	2
1.3	Variable Importance: le due definizioni classiche	2
1.3.1	Mean Decrease Impurity (MDI)	2
1.3.2	Mean Decrease Accuracy (MDA / Permutation Importance)	2
1.4	Il bias della MDI: perché favorisce certe variabili	3
1.5	Il problema della correlazione e l'instabilità	3
1.6	Soluzioni: Conditional Importance	3
1.6.1	Conditional Permutation Importance (Strobl et al., 2008)	3
2	Setup	4
3	Esperimento 1 — Bias della MDI verso variabili ad alta cardinalità	5
4	Esperimento 2 — Instabilità della MDI in presenza di correlazione	7
4.1	Setup della simulazione	7
4.2	Instabilità: il ranking MDI cambia tra run	8
5	Esperimento 3 — MDI vs Permutation vs Conditional Importance	10
5.1	Confronto diretto sui dati correlati	10
6	Esperimento 4 — Confronto quantitativo: precision e recall	12
6.1	Setup con struttura a blocchi	12
6.2	Precision e Recall su 40 run	13
7	Riepilogo e confronto sintetico	15
8	Riferimenti	16

1 Fondamenti teorici

1.1 Il contesto: perché questo problema è rilevante

Immagina di avere un dataset medico con migliaia di variabili — per esempio i livelli di espressione di 10.000 geni in un gruppo di pazienti — e di voler capire quali di questi geni sono effettivamente collegati alla sopravvivenza o alla risposta a un farmaco.

Non puoi analizzarli tutti uno a uno: hai bisogno di uno strumento che ti dica automaticamente “questi 20 geni sono quelli che contano, ignora gli altri 9.980”. Uno degli strumenti più usati per questo è il **Random Forest**, che oltre a fare previsioni produce anche una classifica di importanza per ogni variabile. In teoria, le variabili in cima alla classifica sono quelle più utili per spiegare il fenomeno.

Il problema è che questa classifica è spesso **inaffidabile**, e questo notebook ne mostra i motivi:

- L'indice di importanza più comune (MDI) favorisce sistematicamente certe variabili, anche quando queste non hanno alcun legame reale con quello che stai cercando di prevedere.
- Quando alcune variabili sono simili tra loro (correlate), il modello distribuisce il “credito” in modo arbitrario, rendendo i risultati caotici e instabili.

1.2 Random Forest: una sintesi

Un **Random Forest** è un insieme (*ensemble*) di tanti alberi decisionali che lavorano insieme. Un singolo albero tende a “memorizzare” troppo i dati su cui è stato addestrato (overfitting). Il Random Forest risolve questo costruendo centinaia di alberi diversi e facendoli “votare” insieme, grazie a due meccanismi di casualizzazione:

1. **Bootstrap**: ogni albero è addestrato su un campione estratto con reimmissione dal dataset originale. Le osservazioni non estratte si chiamano **OOB** (Out-Of-Bag) e fungono da insieme di validazione interno automatico.
2. **Selezione casuale delle variabili**: ad ogni split ogni albero considera solo un sottoinsieme casuale di $m = \lfloor \sqrt{p} \rfloor$ variabili, rendendo gli alberi sufficientemente diversi tra loro da ridurre la varianza dell'insieme.

1.3 Variable Importance: le due definizioni classiche

1.3.1 Mean Decrease Impurity (MDI)

La MDI, anche detta *Gini Importance*, misura quanto ogni variabile ha contribuito a ridurre il disordine (impurità) nei nodi degli alberi:

$$\text{MDI}(X_j) = \frac{1}{B} \sum_{b=1}^B \sum_{\substack{t \in T_b \\ v(t)=j}} p_b(t) \cdot \Delta i(t, X_j)$$

dove $p_b(t) = n_t/n$ è la proporzione di osservazioni che arrivano al nodo t nell'albero b e $\Delta i(t, X_j)$ è la riduzione di impurità prodotta dallo split su X_j in quel nodo.

Vantaggio: si calcola a costo zero durante l'addestramento.

Svantaggio fondamentale: è sistematicamente distorta verso variabili con molti valori distinti (alta cardinalità), come vedremo nell'Esperimento 1.

1.3.2 Mean Decrease Accuracy (MDA / Permutation Importance)

Proposta da Breiman (2001), risponde a una domanda più intuitiva: cosa succede alle previsioni se rendo una variabile completamente inutile? Per ogni X_j :

1. Calcola l'errore OOB di ogni albero ($\text{err}_0^{(b)}$).
2. Rimescola casualmente i valori di X_j nelle osservazioni OOB, distruggendone l'informazione, e ricalcola l'errore ($\text{err}_j^{(b)}$).
3. La MDA è la media del peggioramento:

$$\text{MDA}(X_j) = \frac{1}{B} \sum_{b=1}^B [\text{err}_j^{(b)} - \text{err}_0^{(b)}]$$

Se X_j è irrilevante, rimescolarla non cambia le previsioni $\Rightarrow \text{MDA}(X_j) \approx 0$. Se è informativa, rimescolarla peggiora il modello $\Rightarrow$ valore alto.

Limite residuo: la MDA riduce ma non elimina il bias da cardinalità. Rimescolare una variabile continua con molti valori distinti produce coppie (osservazione, valore permutato) che non esistono nel dataset originale, perturbando artificialmente le previsioni OOB. Con n grande e variabili ad alta cardinalità, questo effetto può produrre valori MDA non trascurabili anche per predittori completamente irrilevanti, come mostreremo nell'Esperimento 1.

1.4 Il bias della MDI: perché favorisce certe variabili

Supponiamo di avere una variabile completamente casuale, priva di qualsiasi legame con Y . Ci aspetteremmo $\text{MDI} \approx 0$. Purtroppo non è così, per due motivi strutturali:

1. Più valori possibili = più opportunità di sembrare utile per caso. Una variabile continua può essere tagliata in centinaia di soglie diverse. Anche se è puro rumore, il fatto di avere molte soglie tra cui scegliere fa sì che per puro caso una sembri utile. Formalmente, il massimo su K tentativi cresce con K anche se ogni tentativo ha guadagno atteso zero:

$$E\left[\max_{k=1}^K \Delta i_k\right] \text{ è strettamente crescente in } K, \quad \text{anche se } \Delta i_k \stackrel{\text{iid}}{\sim} F \text{ con } E[\Delta i_k] = 0$$

Una variabile binaria ha una sola possibile soglia; una variabile continua ne ha potenzialmente centinaia. Quindi risulterà sempre più “importante”, non perché contenga più informazione.

2. La selezione casuale delle variabili amplifica il problema. Poiché ad ogni split si considerano solo $m < p$ variabili, una variabile ad alta cardinalità ha più probabilità di risultare la “migliore” nel sottoinsieme estratto per puro caso.

Conseguenza (Strobl et al., 2007):

$$E[\text{MDI}(X_j)] > 0 \quad \text{anche se } X_j \perp Y$$

e questo bias cresce al crescere del numero di valori distinti.

1.5 Il problema della correlazione e l'instabilità

Quando il Random Forest incontra due variabili molto correlate, entrambe funzionano quasi ugualmente bene per fare uno split. Quale delle due “vince” il nodo dipende da fattori puramente casuali: quali osservazioni sono nel bootstrap, quale sottoinsieme di variabili è stato estratto.

Il risultato è che l'importanza viene distribuita arbitrariamente tra le variabili correlate, e questa distribuzione cambia ad ogni esecuzione del modello:

Correndo lo stesso Random Forest due volte sugli stessi dati, la classifica delle variabili importanti può cambiare sostanzialmente. Questo è il problema dell'**instabilità**.

In un contesto reale — dove si vogliono identificare le variabili davvero causali — questa instabilità è pericolosa: un analista che selezionasse le prime k variabili potrebbe includere variabili rumore diverse ad ogni esecuzione, e il “gene più importante” di oggi potrebbe non comparire nella lista di domani.

1.6 Soluzioni: Conditional Importance

1.6.1 Conditional Permutation Importance (Strobl et al., 2008)

La Permutation Importance standard rimescola X_j ignorando le variabili correlate, creando combinazioni di valori impossibili che generano un trasferimento artificiale di importanza tra variabili correlate.

L'idea di Strobl et al. è di rimescolare X_j solo all'interno di gruppi di osservazioni con valori simili per le variabili correlate. In questo modo l'importanza misurata riflette il contributo unico di X_j , al netto di quello già spiegato dalle variabili correlate. Il gruppo di correlazione è: $\mathcal{Z}_j = \{X_k : |\hat{\rho}(X_j, X_k)| > \lambda\}$.

Nota importante: in R il pacchetto `party::cforest` non è semplicemente un diverso indice sullo stesso modello. Usa conditional inference trees (Hothorn et al., 2006), in cui gli split sono scelti tramite test di permutazione invece del criterio Gini. Questo elimina già alla base il bias da cardinalità, *prima* ancora di applicare la permutazione condizionale. Il confronto con `randomForest` è quindi tra due modelli con algoritmi di splitting diversi, non solo tra due indici.

2 Setup

```
pkgs <- c("randomForest", "party",
          "MASS", "ggplot2", "dplyr", "tidyr", "patchwork",
          "reshape2", "RColorBrewer", "forcats", "scales")
to_install <- pkgs[!pkgs %in% installed.packages()[, "Package"]]

repos <- getOption("repos")
if (is.null(repos) || identical(repos[["CRAN"]], "@CRAN@") || is.na(repos[["CRAN"]])) {
  repos[["CRAN"]] <- "https://cloud.r-project.org"
  options(repos = repos)
}
if (length(to_install)) install.packages(to_install, dependencies = TRUE)

library(randomForest)
library(party)
library(MASS)
library(ggplot2)
library(dplyr)
library(tidyr)
library(patchwork)
library(reshape2)
library(RColorBrewer)
library(forcats)
library(scales)

tema_base <- theme_minimal(base_size = 12) +
  theme(
    plot.title      = element_text(face = "bold", size = 13),
    plot.subtitle   = element_text(color = "grey40", size = 10),
    legend.position = "bottom",
    axis.text       = element_text(size = 10),
    plot.margin     = margin(8, 8, 8, 8)
  )
theme_set(tema_base)

set.seed(42)
```

3 Esperimento 1 — Bias della MDI verso variabili ad alta cardinalità

Obiettivo: verificare sperimentalmente che la MDI assegna importanza crescente al crescere del numero di valori distinti, anche in assenza di qualsiasi segnale.

Disegno: Y è puro rumore casuale; i quattro predittori sono anch'essi rumore puro, ma con cardinalità crescente: binaria (2 valori), categoriale a 5 livelli, categoriale a 15 livelli, e continua. Poiché nessuna variabile contiene informazione, qualsiasi indice corretto dovrebbe assegnare importanza vicina a zero a tutte.

```
n <- 600
set.seed(101)

df_bias <- data.frame(
  Y      = rnorm(n),
  X_bin  = factor(sample(c("A","B"),      n, replace = TRUE)),
  X_cat5 = factor(sample(paste0("L", 1:5), n, replace = TRUE)),
  X_cat15 = factor(sample(paste0("L", 1:15), n, replace = TRUE)),
  X_cont = rnorm(n)
)

set.seed(101)
rf_bias <- randomForest(Y ~ ., data = df_bias, ntree = 1000, importance = TRUE)

imp_mdi <- importance(rf_bias, type = 2)[, 1]
imp_mda <- importance(rf_bias, type = 1)[, 1]

var_labels <- c(
  X_bin  = "X bin\n(2 cat)",
  X_cat5 = "X cat5\n(5 cat)",
  X_cat15 = "X cat15\n(15 cat)",
  X_cont = "X cont\n(continua)"
)

imp_df <- data.frame(
  Variable = factor(var_labels[names(imp_mdi)]), levels = var_labels,
  MDI      = imp_mdi,
  Permutation = imp_mda
)

# Stampa valori numerici per verifica
cat("Valori MDI:\n");      print(round(imp_mdi, 2))

## Valori MDI:
##   X_bin  X_cat5 X_cat15  X_cont
##   6.07   22.70   59.09   67.05

cat("\nValori Permutation:\n"); print(round(imp_mda, 2))

##
## Valori Permutation:
##   X_bin  X_cat5 X_cat15  X_cont
##  -2.75    6.09    7.21   14.04
```

```

# --- Dati in formato lungo per barre raggruppate ---
imp_long <- imp_df %>%
  pivot_longer(c(MDI, Permutation),
    names_to = "Metodo",
    values_to = "Valore") %>%
  mutate(Metodo = factor(Metodo, levels = c("MDI", "Permutation")))

# Moltiplicatori MDI rispetto alla variabile binaria (per annotazioni)
mdi_ref <- imp_mdi["X_bin"]
mdi_mult <- round(imp_mdi / mdi_ref, 1)
mult_labels <- paste0("\u00d7", mdi_mult) # *1, *3.7, ...

# Posizioni x delle barre MDI nel grafico dodge (leggermente a sinistra del centro)
var_levels <- levels(imp_long$Variable)
x_pos_mdi <- seq_along(var_levels) - 0.2

annot_df <- data.frame(
  x_dodge = x_pos_mdi,
  y_top = as.numeric(imp_mdi) + 2.5,
  label = mult_labels
)

# Palette
pal_metodi <- c(MDI = "#E63946", Permutation = "#457B9D")

ggplot(imp_long,
  aes(x = Variable, y = Valore, fill = Metodo, group = Metodo)) +
  # Barre raggruppate
  geom_col(position = position_dodge(0.72), width = 0.65, alpha = 0.92) +
  # Linea "valore atteso corretto = 0"
  geom_hline(yintercept = 0, color = "#2d6a4f", linewidth = 1) +
  # Etichetta della linea
  annotate("text", x = 4.35, y = 2.5,
    label = "Atteso: 0", color = "#2d6a4f",
    size = 3, hjust = 0, fontface = "italic") +
  # Moltiplicatori sopra le barre MDI
  geom_text(data = annot_df,
    aes(x = x_dodge, y = y_top, label = label),
    inherit.aes = FALSE,
    color = "#B5002A", fontface = "bold", size = 3.6, vjust = 0) +
  scale_fill_manual(values = pal_metodi,
    labels = c("MDI", "MDA")) +
  scale_y_continuous(expand = expansion(mult = c(0, 0.18))) +
  coord_cartesian(clip = "off") +
  labs(
    title = "Bias della MDI verso variabili ad alta cardinalit\u00e0",
    subtitle = "Y \u00e8 rumore puro \u2014 nessun predittore ha alcun legame con Y",
    x = "Predittore (da bassa a alta cardinalit\u00e0)",
    y = "Importanza stimata (valori relativi)",
    fill = NULL,
    caption = ""
  ) +
  theme(

```

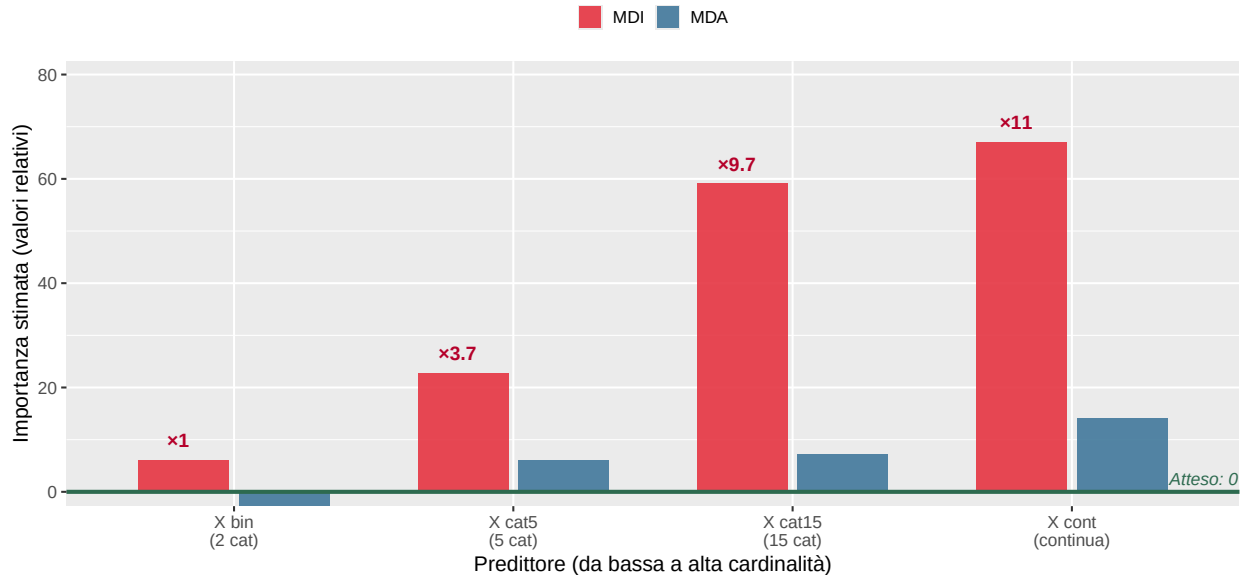
```

legend.position      = "top",
legend.key.size      = unit(0.45, "cm"),
plot.caption        = element_text(color = "grey45", size = 8.5, hjust = 0),
plot.caption.position = "plot"
)

```

Bias della MDI verso variabili ad alta cardinalità

Y è rumore puro — nessun predittore ha alcun legame con Y



Interpretazione: La MDI (barre rosse) mostra chiaramente un'inflazione dell'importanza al crescere della cardinalità: tutte le variabili sono rumore, eppure la variabile continua risulta molto più importante della binaria. I moltiplicatori sopra le barre MDI (ad es. $\times 1$, $\times 3.7$, $\times 9.7$, $\times 11$) quantificano questa inflazione rispetto alla variabile binaria di riferimento.

La MDA (barre blu, Permutation) è molto più vicina allo zero e fornisce una misura più conservativa della rilevanza: permutare una variabile irrilevante dovrebbe lasciare invariate le prestazioni, quindi un valore vicino a zero è atteso. Tuttavia la Permutation non è completamente immune agli artefatti di cardinalità: variabili ad alta cardinalità possono ancora mostrare un contributo positivo, come si vede per **X_cont**.

La linea **Atteso: 0** indica il valore teorico per un predittore irrilevante. In sintesi: - la MDI è fortemente distorta dalla cardinalità e tende a sovrastimare variabili con molti valori distinti; - la MDA mitiga gran parte di questo bias, ma può conservare residui di importanza per predittori ad alta cardinalità; Per ridurre definitivamente il bias è preferibile usare misure condizionate (p.es. Conditional Importance tramite **cforest**), che vedremo nell'Esperimento 3.

4 Esperimento 2 — Instabilità della MDI in presenza di correlazione

4.1 Setup della simulazione

Creiamo un dataset con $p = 12$ variabili con struttura **AR(1)** a $\rho = 0.95$ — correlazione molto forte, in modo che le variabili vicine siano quasi indistinguibili dal punto di vista predittivo. Il segnale causale è deliberatamente debole:

$$Y = 0.8 \cdot X_1 + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1), \quad \mathbf{X} \sim \mathcal{N}(\mathbf{0}, \Sigma_{\rho}^{\text{AR}(1)})$$

Con $\beta_1 = 0.8$ e $n = 150$ osservazioni, il rapporto segnale/rumore è basso: un metodo instabile fatica a distinguere X_1 dalle sue vicine correlate.

```
n <- 150
p <- 12
rho <- 0.95

Sigma <- toeplitz(rho^(0:(p-1)))
set.seed(202)
X_mat <- mvrnorm(n, mu = rep(0, p), Sigma = Sigma)
colnames(X_mat) <- paste0("X", 1:p)

beta_true <- c(0.8, rep(0, p - 1))
Y_corr <- as.numeric(X_mat %*% beta_true + rnorm(n))
df_corr <- data.frame(Y = Y_corr, X_mat)
```

4.2 Instabilità: il ranking MDI cambia tra run

Eseguiamo lo stesso Random Forest 30 volte cambiando solo il seme casuale (non i dati) e registriamo il rango assegnato dalla MDI a ciascuna variabile (1 = più importante). Stampiamo la tabella completa di medie e deviazioni standard per ogni variabile.

```
n_runs <- 30
rank_matrix <- matrix(NA, nrow = p, ncol = n_runs,
                      dimnames = list(paste0("X", 1:p), paste0("Run", 1:n_runs)))

for (i in seq_len(n_runs)) {
  set.seed(i * 13)
  rf_i <- randomForest(Y ~ ., data = df_corr, ntree = 300)
  rank_matrix[, i] <- rank(-importance(rf_i, type = 2)[, 1])
}

# Tabella sintetica: rango medio e SD per ciascuna variabile
rank_summary <- apply(rank_matrix, 1, function(x)
  c(media = round(mean(x), 1), sd = round(sd(x), 2),
    min = min(x), max = max(x)))
print(t(rank_summary))
```

```
##      media    sd min max
## X1      1.0 0.00   1   1
## X2      2.0 0.00   2   2
## X3      3.1 0.43   3   5
## X4      4.0 0.41   3   5
## X5      9.7 1.42   6  12
## X6      6.5 1.01   4   9
## X7      6.4 1.35   4   9
## X8      5.7 1.15   4  10
## X9      9.7 1.52   6  12
## X10     11.6 0.72   9  12
## X11      9.3 1.69   6  12
## X12      9.0 1.35   7  12
```



```

rank_long <- as.data.frame(rank_matrix) %>%
  tibble::rownames_to_column("Variable") %>%
  pivot_longer(-Variable, names_to = "Run", values_to = "Rank") %>%
  mutate(Variable = factor(Variable, levels = paste0("X", 1:p)))

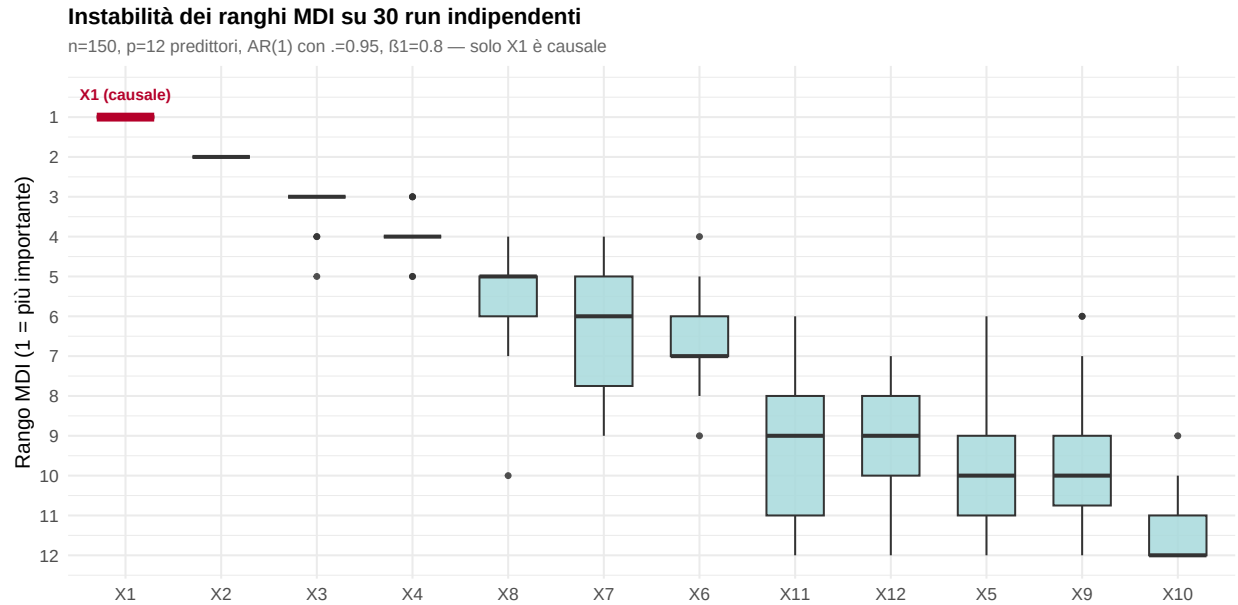
# Ordine sull'asse x: variabili ordinate per rango mediano
var_order <- rank_long %>%
  group_by(Variable) %>%
  summarise(med = median(Rank), .groups = "drop") %>%
  arrange(med) %>%
  pull(Variable) %>%
  as.character()

rank_long <- rank_long %>%
  mutate(Variable = factor(as.character(Variable), levels = var_order))

# Posizione x di X1 nell'ordine del grafico (per l'annotazione)
x1_pos <- which(var_order == "X1")

ggplot(rank_long, aes(x = Variable, y = Rank)) +
  # Tutti i box in azzurro
  geom_boxplot(fill = "#A8DADC", alpha = 0.85, outlier.size = 1.2, width = 0.6) +
  # Sovrapponi il box di X1 in rosso con bordo marcato
  geom_boxplot(
    data = dplyr::filter(rank_long, Variable == "X1"),
    fill = "#E63946", alpha = 0.9, width = 0.6,
    color = "#B5002A", linewidth = 1.2
  ) +
  # Etichetta testuale sopra X1
  annotate("text", x = x1_pos, y = 0.3,
    label = "X1 (causale)", color = "#B5002A",
    size = 3.2, fontface = "bold", vjust = 1) +
  scale_y_reverse(breaks = 1:p) +
  labs(
    title = "Instabilit\u00e0 dei ranghi MDI su 30 run indipendenti",
    subtitle = paste0("n=", n, ", p=", p, " predittori, AR(1) con \u03c1=", rho,
      ", \u03b2\u2081=0.8 \u2014 solo X1 \u00e8 causale"),
    x = NULL, y = "Rango MDI (1 = pi\u00f9 importante)"
  )

```



Lettura del grafico e della tabella: Ogni box mostra la distribuzione dei ranghi di una variabile nelle 30 esecuzioni.

Il risultato rivela due livelli distinti di instabilità. In cima, X_1 e X_2 mantengono le prime due posizioni in modo quasi stabile (SD bassa): la MDI riesce a separarle dal resto del gruppo. Il vero problema però emerge nelle posizioni successive: le variabili X_3 – X_{12} si contendono le posizioni 3–12 in modo fortemente caotico, con SD molto elevata. Per un analista reale che non conosce la verità e selezionasse le prime $k = 5$ variabili, il set selezionato cambierebbe sostanzialmente ad ogni esecuzione: al posto di rumore ordinato troverebbe rumore diverso ogni volta. Questo è il danno concreto dell'instabilità: non necessariamente perdere X_1 , ma non poter sapere *quali* variabili di corredo sono affidabili.

5 Esperimento 3 — MDI vs Permutation vs Conditional Importance

5.1 Confronto diretto sui dati correlati

Confrontiamo i tre indici sugli stessi dati AR(1), normalizzando i valori tra 0 e 1. Si ricordi che la Conditional Importance è calcolata tramite `cforest`, che usa un algoritmo di splitting diverso (conditional inference trees), quindi il confronto è tra approcci differenti, non solo tra metriche diverse sullo stesso modello.

```
# randomForest: MDI e Permutation
set.seed(303)
rf_main      <- randomForest(Y ~ ., data = df_corr, ntree = 800, importance = TRUE)
imp_mdi_main <- importance(rf_main, type = 2)[, 1]
imp_mda_main <- importance(rf_main, type = 1)[, 1]

# cforest: Conditional Importance (Strobl et al. 2008)
# Nota: cforest usa conditional inference trees, non CART -
# il bias da cardinalità è già eliminato a livello di splitting.
set.seed(303)
cf_main <- cforest(
  Y ~ ., data = df_corr,
```

```

    controls = cforest_unbiased(ntree = 800, mtry = floor(sqrt(p)))
  )
imp_cond_main <- varimp(cf_main, conditional = TRUE)

normalize <- function(x) {
  x <- x - min(x)
  if (max(x) == 0) return(x)
  x / max(x)
}

imp_comparison <- data.frame(
  Variable = paste0("X", 1:p),
  MDI = normalize(imp_mdi_main),
  Permutation = normalize(imp_mda_main),
  Conditional = normalize(imp_cond_main[paste0("X", 1:p)])
) %>%
pivot_longer(-Variable, names_to = "Metodo", values_to = "Importanza") %>%
mutate(
  Metodo = factor(Metodo, levels = c("MDI", "Permutation", "Conditional")),
  Causale = Variable == "X1",
  Variable = factor(Variable, levels = paste0("X", 1:p))
)

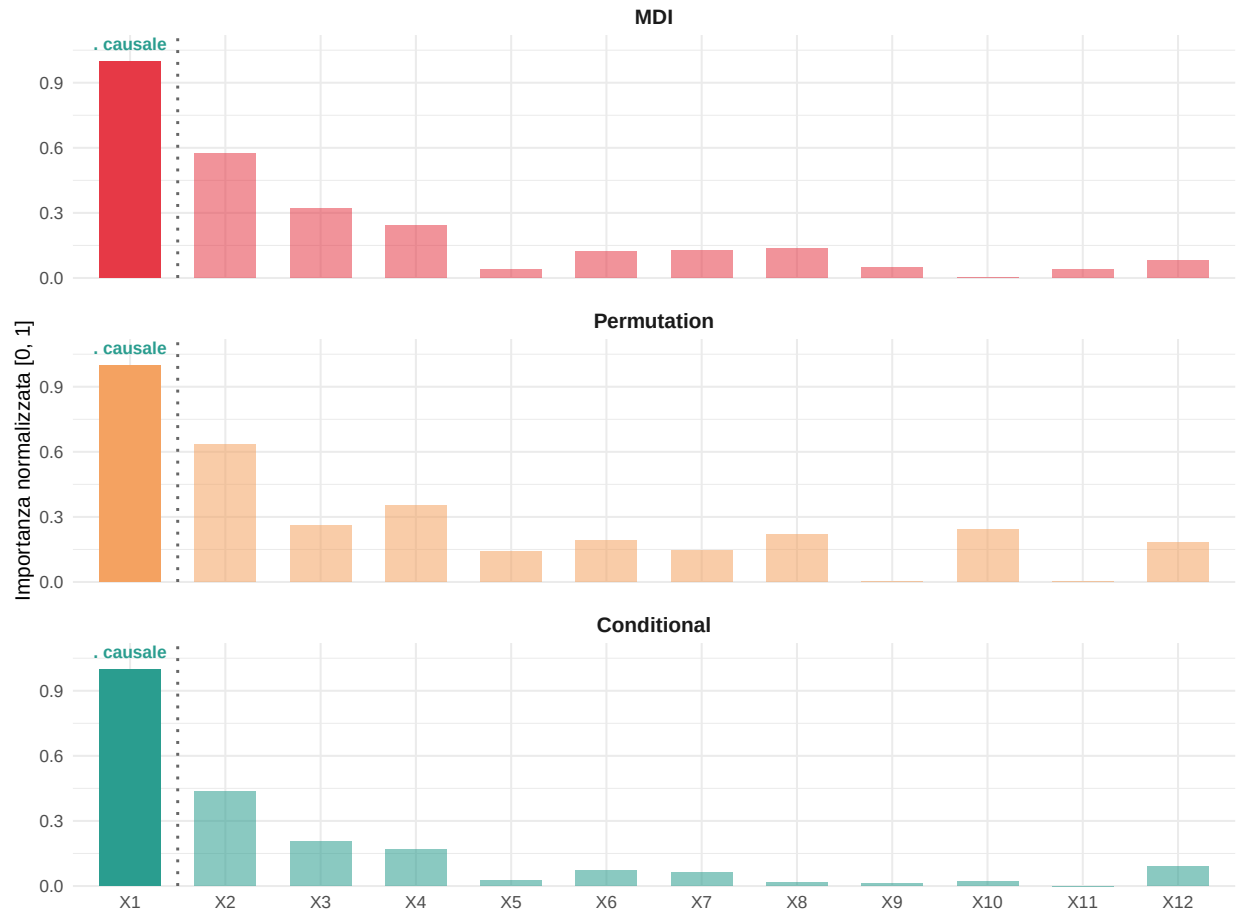
pal3 <- c(MDI = "#E63946", Permutation = "#F4A261", Conditional = "#2A9D8F")

ggplot(imp_comparison,
  aes(x = Variable, y = Importanza, fill = Metodo, alpha = Causale)) +
  geom_col(position = position_dodge(0.75), width = 0.65) +
  geom_vline(xintercept = 1.5, linetype = "dotted", color = "grey40", linewidth = 0.8) +
  scale_fill_manual(values = pal3) +
  scale_alpha_manual(values = c("TRUE" = 1, "FALSE" = 0.55), guide = "none") +
  scale_y_continuous(limits = c(0, 1.12), expand = c(0, 0)) +
  coord_cartesian(clip = "off") +
  annotate("text", x = 1, y = 1.08, label = "\u2713 causale",
    size = 3.5, color = "#2A9D8F", fontface = "bold") +
  facet_wrap(~ Metodo, ncol = 1) +
  labs(
    title = "Confronto tra indici di importanza",
    subtitle = paste0("Solo X1 \u00e8 causale \u2014 dati AR(1) con \u03c1=", rho),
    x = NULL, y = "Importanza normalizzata [0, 1]"
  ) +
  theme(
    legend.position = "none",
    strip.text = element_text(face = "bold", size = 12),
    panel.spacing = unit(1, "lines")
  )
)

```

Confronto tra indici di importanza

Solo X_1 è causale — dati AR(1) con $\rho=0.95$



Lettura del grafico:

Metodo	Comportamento osservato
MDI	Distribuisce l'importanza sul blocco correlato: X_2 è a circa 0.6, X_3 e X_4 intorno a 0.3. Il profilo scende rapidamente dopo X_4
Permutation	Il profilo è simile alla MDI per le prime variabili ($X_2 \approx 0.6$, $X_3-X_4 \approx 0.3$), ma distribuisce importanza residua su un numero maggiore di variabili lontane (X_5-X_{12} tutte con barre visibili)
Conditional	Concentra quasi tutta l'importanza su X_1 (≈ 0.95); X_2 scende a ≈ 0.35 , tutto il resto cade vicino a zero. È l'unico metodo che isola la variabile causale in modo netto in questa singola esecuzione

6 Esperimento 4 — Confronto quantitativo: precision e recall

6.1 Setup con struttura a blocchi

Misuriamo quantitativamente le prestazioni di selezione su $N = 40$ ripetizioni, calcolando per ogni metodo:

- **Precision** = quante delle variabili selezionate sono davvero causali?

- **Recall** = quante delle variabili causali vengono trovate?

Il dataset ha $p = 30$ predittori in **3 blocchi da 10**, con correlazione intra-blocco $\rho_{\text{intra}} = 0.80$ e correlazione inter-blocco $\rho_{\text{inter}} = 0.30$. La correlazione inter-blocco non nulla rende il problema realisticamente difficile: le variabili causali di blocchi diversi non sono più indipendenti tra loro. Le variabili causali sono X_1 , X_{11} , X_{21} (la prima di ogni blocco), con segnale debole $\beta = 1$ e $n = 100$ osservazioni.

```
p6 <- 30
n6 <- 100      # n piccolo: il problema è difficile
rho6_intra <- 0.80
rho6_inter <- 0.30 # correlazione inter-blocco non nulla
beta6_val <- 1    # segnale debole
true_vars6 <- c("X1", "X11", "X21")

beta6 <- rep(0, p6)
beta6[c(1, 11, 21)] <- beta6_val

# Matrice di correlazione a blocchi con correlazione inter-blocco
Sigma6 <- matrix(rho6_inter, p6, p6)      # base: rho_inter ovunque
for (b in 0:2) {
  idx <- (b * 10 + 1):((b + 1) * 10)
  Sigma6[idx, idx] <- rho6_intra          # intra-blocco più alta
}
diag(Sigma6) <- 1

set.seed(600)
X6 <- mvrnorm(n6, mu = rep(0, p6), Sigma = Sigma6)
colnames(X6) <- paste0("X", 1:p6)
Y6 <- as.numeric(X6 %*% beta6 + rnorm(n6))
df6 <- data.frame(Y = Y6, X6)
```

6.2 Precision e Recall su 40 run

```
n_rep6 <- 40
k6 <- 3    # selezioniamo le top-3 variabili

pr_results6 <- lapply(seq_len(n_rep6), function(i) {
  set.seed(i * 17)
  rf6 <- randomForest(Y ~ ., data = df6, ntree = 500, importance = TRUE)

  top_mdi <- names(sort(importance(rf6, type = 2)[, 1], decreasing = TRUE))[1:k6]
  top_perm <- names(sort(importance(rf6, type = 1)[, 1], decreasing = TRUE))[1:k6]

  data.frame(
    run      = i,
    MDI_prec = mean(top_mdi %in% true_vars6),
    Perm_prec = mean(top_perm %in% true_vars6),
    MDI_rec  = sum(true_vars6 %in% top_mdi) / length(true_vars6),
    Perm_rec  = sum(true_vars6 %in% top_perm) / length(true_vars6)
  )
})
pr_df6 <- do.call(rbind, pr_results6)

# Verifica numerica: medie e SD su 40 run
```

```

cat("Medie su 40 run:\n")

## Medie su 40 run:
print(round(colMeans(pr_df6[, -1]), 3))

##   MDI_prec Perm_prec   MDI_rec Perm_rec
##   0.667     0.750     0.667     0.750

cat("\nDeviazioni standard:\n")

##
## Deviazioni standard:
print(round(apply(pr_df6[, -1], 2, sd), 3))

##   MDI_prec Perm_prec   MDI_rec Perm_rec
##   0.000     0.146     0.000     0.146

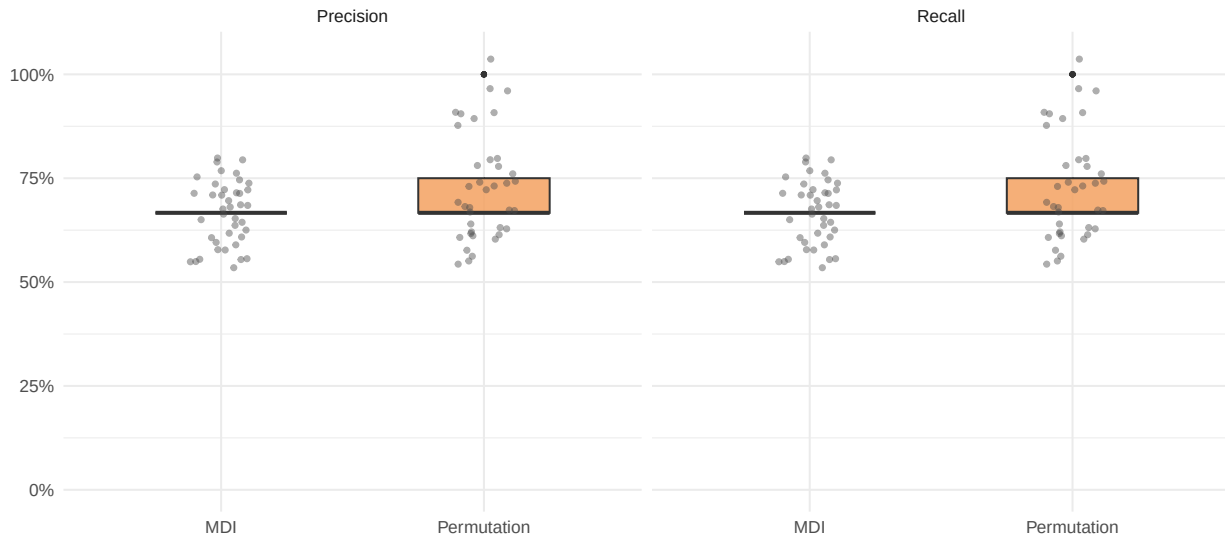
pr_long6 <- pr_df6 %>%
  pivot_longer(-run, names_to = "key", values_to = "value") %>%
  mutate(
    Metodo = ifelse(grepl("MDI", key), "MDI", "Permutation"),
    Metrica = ifelse(grepl("prec", key), "Precision", "Recall")
  )

ggplot(pr_long6, aes(x = Metodo, y = value, fill = Metodo)) +
  geom_boxplot(alpha = 0.85, width = 0.5, outlier.size = 1) +
  geom_jitter(width = 0.12, size = 1.2, alpha = 0.45, color = "grey30") +
  facet_wrap(~ Metrica) +
  scale_fill_manual(values = c(MDI = "#E63946", Permutation = "#F4A261"),
                    guide = "none") +
  scale_y_continuous(limits = c(0, 1.05),
                    labels = percent_format(accuracy = 1)) +
  labs(
    title = "Precision e Recall su 40 run indipendenti",
    subtitle = paste0("3 variabili causali su ", p6,
                      " - blocchi \u03c31=", rho6_intra,
                      ", inter-blocco \u03c31=", rho6_inter,
                      ", \u03b22=", beta6_val, ", n=", n6),
    x = NULL, y = NULL
  )

```

Precision e Recall su 40 run indipendenti

3 variabili causali su 30 — blocchi $\rho=0.8$, inter-blocco $\rho=0.3$, $\beta=1$, $n=100$



Interpretazione: Con segnale debole, n piccolo e correlazione sia intra- che inter-blocco, la differenza tra MDI e Permutation Importance emerge chiaramente sia in precision che in recall.

La MDI mostra $SD=0$ su tutti i 40 run: non è instabile, è **sistematicamente bloccata** su una selezione errata. I punti si concentrano su soli due valori discreti — 33% e 67% di 3 variabili causali, cioè 1 o 2 causali trovate su 3 — e il risultato non cambia mai tra un run e l'altro. Il bias da cardinalità e dalla correlazione intra-blocco porta MDI a selezionare ripetutamente le stesse variabili rumore del blocco (es. X_2 al posto di X_1), producendo errori sistematici e non casuali.

La Permutation Importance ha mediana più alta (~75% vs ~67% di MDI) e variabilità reale tra run ($SD=0.146$): in molte esecuzioni riesce a trovare tutte e 3 le causali (100%), in altre ne manca una. Questo trade-off — più accurata in media, ma con variabilità residua — è il comportamento atteso di un metodo meno distorto ma non esente da errori con n piccolo.

7 Riepilogo e confronto sintetico

```
X1_MDI_top <- which.max(imp_mdi_main) == 1
X1_MDA_top <- which.max(imp_mda_main) == 1
X1_COND_top <- which.max(imp_cond_main[paste0("X", 1:p)]) == 1

# MDI: identificata, ma instabile tra run (nota esplicita)
mdi_label <- if (X1_MDI_top) "S\u00ec \u2714 (instabile)" else "No \u2718"

tick_cross <- function(x) ifelse(x, "S\u00ec \u2714", "No \u2718")

summary_df <- data.frame(
  Metodo = c(
    "MDI (randomForest)",
    "Permutation Imp. (randomForest)",
    "Conditional Imp. (cforest)"
  ),
  Bias_card = c("S\u00ec", "No", "No"),
```

```

Instabile   = c("S\u00ec", "Parzialmente", "No"),
X1_corretta = c(
  mdi_label,
  tick_cross(X1_MDA_top),
  tick_cross(X1_COND_top)
),
Interp_obs  = c("No", "No", "No"),
Costo       = c("Basso", "Medio", "Alto*")
)

knitr::kable(
  summary_df,
  col.names = c("Metodo", "Bias cardinalit\u00e0",
    "Instabile con correlazione",
    "X1 identificata",
    "Interpret. per osservazione",
    "Costo computazionale"),
  align     = "lcccc",
  caption = paste0(
    "Confronto tra metodi di variable importance. ",
    "*cforest usa conditional inference trees, algoritmo di splitting diverso da randomForest."
  )
)

```

Table 1: Confronto tra metodi di variable importance. *cforest usa conditional inference trees, algoritmo di splitting diverso da randomForest.

Metodo	Bias cardinalità	Instabile con correlazione	X1 identificata	Interpret. per osservazione	Costo computazionale
MDI (randomForest)	Sì	Sì	Sì (instabile)	No	Basso
Permutation Imp. (randomForest)	No	Parzialmente	Sì	No	Medio
Conditional Imp. (cforest)	No	No	Sì	No	Alto*

Lettura della tabella: Non esiste un metodo dominante su tutte le dimensioni. La scelta dipende dal contesto:

- Se il **costo computazionale** è un vincolo stringente e i predittori non sono ad alta cardinalità, la **Permutation Importance** è un buon compromesso.
- Se si lavora con predittori fortemente correlati e si vuole individuare la causa diretta, la **Conditional Importance** è la scelta più rigorosa — ricordando che si tratta di un modello diverso (cforest), non solo di un indice diverso su randomForest.

8 Riferimenti

- Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
- Strobl, C., Boulesteix, A.-L., Zeileis, A., Hothorn, T. (2007). *Bias in Random Forest Variable Importance Measures: Illustrations, Sources and a Solution*. BMC Bioinformatics, 8, 25.

- Strobl, C., Boulesteix, A.-L., Kneib, T., Augustin, T., Zeileis, A. (2008). *Conditional Variable Importance for Random Forests*. BMC Bioinformatics, 9, 307.
- Hothorn, T., Hornik, K., Zeileis, A. (2006). *Unbiased Recursive Partitioning: A Conditional Inference Framework*. Journal of Computational and Graphical Statistics, 15(3).

```
## R version 4.5.2 (2025-10-31 ucrt)
## Platform: x86_64-w64-mingw32/x64
## Running under: Windows 11 x64 (build 26200)
##
## Matrix products: default
##   LAPACK version 3.12.1
##
## locale:
## [1] LC_COLLATE=Italian_Italy.utf8  LC_CTYPE=Italian_Italy.utf8
## [3] LC_MONETARY=Italian_Italy.utf8 LC_NUMERIC=C
## [5] LC_TIME=Italian_Italy.utf8
##
## time zone: Europe/Rome
## tzcode source: internal
##
## attached base packages:
## [1] stats4      grid        stats      graphics  grDevices  utils      datasets
## [8] methods     base
##
## other attached packages:
## [1] scales_1.4.0      forcats_1.0.1      RColorBrewer_1.1-3
## [4] reshape2_1.4.5    patchwork_1.3.2     tidyr_1.3.2
## [7] dplyr_1.2.0        ggplot2_4.0.2       MASS_7.3-65
## [10] party_1.3-20       strucchange_1.5-4    sandwich_3.1-1
## [13] zoo_1.8-15         modeltools_0.2-24    mvtnorm_1.3-7
## [16] randomForest_4.7-1.2
##
## loaded via a namespace (and not attached):
## [1] generics_0.1.4     coin_1.4-3          stringi_1.8.7       lattice_0.22-7
## [5] digest_0.6.39      magrittr_2.0.4      evaluate_1.0.5      fastmap_1.2.0
## [9] plyr_1.8.9         Matrix_1.7-4        survival_3.8-3      multcomp_1.4-30
## [13] purrr_1.2.1        TH.data_1.1-5       codetools_0.2-20    cli_3.6.5
## [17] rlang_1.1.7         splines_4.5.2        withr_3.0.2         yaml_2.3.12
## [21] ote1_0.2.0          tools_4.5.2          parallel_4.5.2      vctrs_0.7.1
## [25] R6_2.6.1            matrixStats_1.5.0    lifecycle_1.0.5     stringr_1.6.0
## [29] libcoin_1.0-12      pkgconfig_2.0.3      pillar_1.11.1       gtable_0.3.6
## [33] Rcpp_1.1.1          glue_1.8.0           xfun_0.56           tibble_3.3.1
## [37] tidyselect_1.2.1    knitr_1.51           farver_2.1.2         htmltools_0.5.9
## [41] labeling_0.4.3      rmarkdown_2.30       compiler_4.5.2       S7_0.2.1
```